Relatório de Auditoria de Segurança

Big Cat Stock Tamer

2 de setembro de 2026

7 achados · **1 crítica** · **3 altas** · **1 média** · **1 baixa** · **1 informativa**
15 pontos fortes verificados

Escopo auditado

Todo o repositório na revisão **1a1a768** do branch **master**: os 58 handlers de rota registrados, os services e schemas de cada módulo, as camadas domain/, integrations/, plugins/, infra/, shared/ e client/, o schema do Prisma e suas migrations, os arquivos de configuração e de deploy (systemd, GitHub Actions, .env.example) e o histórico completo do git. Excluídos da revisão: *src/generated/* (client do Prisma, gerado) e *node_modules/*.

Os achados foram verificados no código real. Os marcados como reproduzidos foram confirmados por requisição contra a aplicação montada com `app.inject()`, e as sondagens usadas foram removidas ao final — a árvore de trabalho ficou limpa.

Nota metodológica: as categorias nesta stack

Antes da revisão, a stack foi detectada a partir de package.json, prisma/schema.prisma, src/app.ts e dos arquivos de deploy. Cada categoria do roteiro foi então traduzida para o equivalente real deste projeto — quando não há equivalente, isso está dito, em vez de forçar um achado.

Stack detectada

Linguagem / runtime	TypeScript 5.9 · Node 22 (ESM)
Framework HTTP	Fastify 5 (+ @fastify/cookie, formbody, multipart, static)
ORM / banco	Prisma 7 · SQLite via @prisma/adapter-better-sqlite3
Autenticação	Usuário único, senha em APP_AUTH_PASSWORD; sessão em cookie + arquivo de hashes
Frontend	JSX renderizado no servidor (preact-render-to-string) + HTMX + Bootstrap 5 via CDN
Client-side	TypeScript compilado por esbuild para public/js/
Validação	Zod 4 na fronteira de rota e de integração
Deploy	systemd (deploy/big-cat.service + big-cat.path) · GitHub Actions (.github/workflows/ci.yml)
Sem Docker/Helm/Terraform	Não há Dockerfile, docker-compose, chart Helm nem módulo Terraform no repositório

Mapeamento das categorias

1. Banco sem tranca (isolamento de inquilino)

O equivalente de RLS nesta stack seria um filtro por dono nas queries do Prisma. O schema (prisma/schema.prisma) foi lido por inteiro: os 11 modelos — AssetClass, Asset, Transaction, BrokerNote, Dividend, PriceHistory, MonthlySnapshot, ExchangeRate, BenchmarkPrice, CdiRate, RiskMetric — não têm coluna de usuário, organização, workspace ou tenant. A aplicação é single-user por desenho (uma senha só, sem tabela de usuários). Categoria NÃO APLICÁVEL: não existe inquilino a isolar. O mecanismo de isolamento real é o portão de sessão único do onRequest em src/plugins/auth.ts:21, auditado na categoria 4.

2. Permissão definida no navegador

Não há sistema de papéis (nenhum isAdmin, canEdit ou role no código). O único gate de recurso na interface é a aba de importação por nota de negociação, escondida quando noteImportEnabled é falso (transaction.routes.ts:107 → views/pages/transactions.tsx). Esse gate FOI cruzado com o backend e tem verificação equivalente no servidor (anthropic.client.ts:110), confirmada por requisição direta. Categoria sem achados.

3. IDOR

Os 58 handlers registrados foram percorridos um a um (não por amostragem): 8 em allocation, 9 em asset, 3 em auth, 11 em dividend, 4 em evolution, 4 em portfolio, 2 em risk, 15 em transaction e 2 em app.ts. Todas as rotas endereçadas por id (/transactions/:id, /dividends/:id, /allocation/classes/:id, /transactions/notes/:id) operam sobre objetos de um único dono. Sem segunda identidade no sistema, não há posse cruzada a violar. Categoria NÃO APLICÁVEL na forma clássica; ver o ponto de atenção registrado como informativo.

4. Chaves expostas

Varridos: código-fonte, prisma/, tests/, docs/, .env.example, .github/workflows/ci.yml, deploy/*.service e deploy/README.md, além do histórico completo do git (git log --all -S e git grep sobre todos os commits, procurando sk-ant-, AKIA e blocos PRIVATE KEY). Nenhum segredo real embutido. O achado desta categoria é o oposto: um default vazio que desliga um controle de segurança inteiro sem validação de boot que o rejeite.

5. Inputs sem tratamento (XSS)

Frontend: procurados innerHTML, outerHTML, insertAdjacentHTML, eval, new Function, document.write e dangerouslySetInnerHTML em src/client/ e src/views/. Backend: procurado input do usuário chegando a resposta HTML sem escape. As views JSX escapam por padrão; o caminho furado é a resposta de erro em texto puro, comprovado por requisição real.

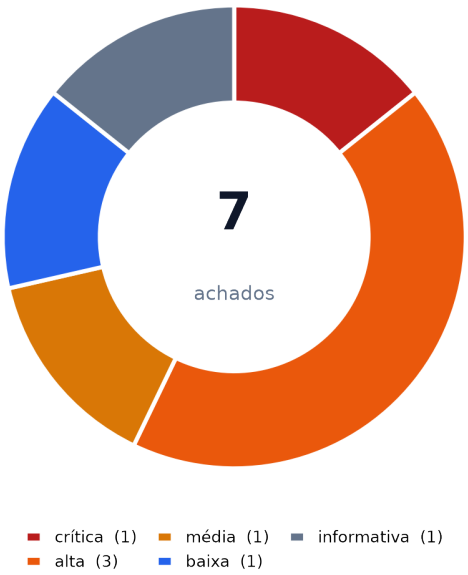
Resumo executivo

A auditoria registrou **7** achados e **15** pontos fortes verificados. O achado crítico é único e tem uma causa só: a autenticação da aplicação pode ficar inteiramente desligada por omissão de uma variável de ambiente, sem que o boot recuse subir. Ele não é apenas o mais grave — é também o que transforma o XSS refletido de um self-XSS num ataque explorável a partir de outro site. Duas das três falhas altas (open redirect e ausência de limite de tentativas no login) são independentes e corrigíveis em poucas linhas cada.

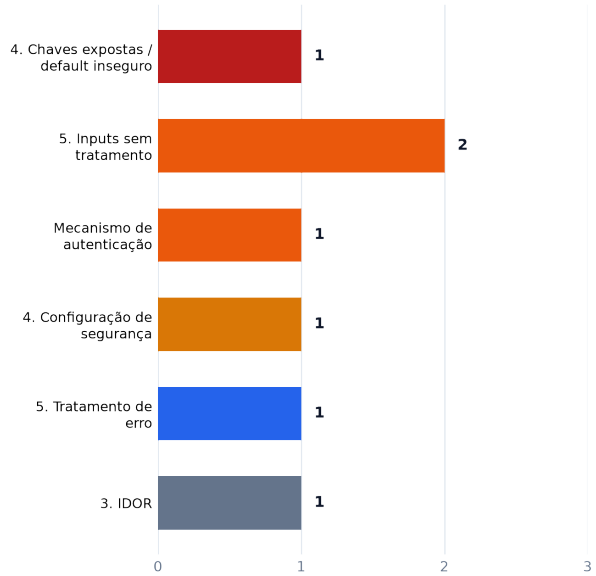
O contraponto importa: o portão de sessão é sólido e resistiu a doze tentativas de bypass por prefixo de URL, não há SQL cru com entrada do usuário, as views escapam por padrão, o download de notas não tem path traversal e não há um único segredo embutido no código, nos configs, no CI ou no histórico do git. As falhas encontradas são de *configuração e de bordas*, não de arquitetura.

1	3	1	1	1	15
CRÍTICA	ALTA	MÉDIA	BAIXA	INFORMATIVA	PONTOS FORTES

Achados por severidade



Achados por categoria



A cor da barra indica a severidade mais grave daquela categoria. As categorias 1 (isolamento de inquilino) e 2 (permissão definida no navegador) não produziram achados e não aparecem no gráfico — o motivo está na nota metodológica.

Pontos fortes verificados

O que segue não é elogio genérico: cada item foi verificado no arquivo e na linha citados, e vários foram confirmados por requisição real contra a aplicação. Esta seção também é a prova de cobertura da auditoria.

OK **1. Portão de sessão sem furo de prefixo**

`src/plugins/auth.ts:21-35`

O hook onRequest é registrado por fastify-plugin, então vale para toda rota registrada depois — inclusive /health e as rotas /api. Testadas 12 variações de bypass contra a lista PUBLIC_PREFIXES (/login/../assets/, /js/../assets/, /js/%2e%2e/assets/, /favicon.ico/../assets/, //assets/, /logout/../assets/, /loginX/assets/, entre outras): todas redirecionaram para /login ou devolveram 404. Nenhuma alcançou conteúdo protegido.

OK **2. Rotas JSON também protegidas**

`src/app.ts:65-72`

Verificado individualmente: /portfolio/api, /assets/api, /allocation/api, /transactions/api, /dividends/api, /evolution/api e /health respondem 302 para /login sem sessão. Não há "porta dos fundos" de API fora do portão.

OK **3. Sessão guardada como hash, comparada em tempo constante**

`src/modules/auth/auth.service.ts:63, 84, 119-129`

O token é randomBytes(32) em base64url; o arquivo auth.key guarda apenas sha256 do token, então vazar o arquivo não dá acesso. A comparação usa timingSafeEqual com checagem prévia de tamanho. Sessões expiradas são descartadas na leitura (auth.service.ts:82).

OK **4. Nenhum SQL cru recebe entrada do usuário**

`src/config/db.ts:18-19, src/app.ts:61, src/infra/backup.ts:88+116`

Só existem três usos de $executeRawUnsafe/$queryRawUnsafe em produção: dois PRAGMA constantes, um SELECT 1 constante e o VACUUM INTO do backup — cujo caminho vem da configuração, não de requisição, e ainda passa por sqlLiteral() (backup.ts:116). Todo o resto do acesso a dados é Prisma parametrizado.

OK **5. Escape por padrão nas views, com um único dangerouslySetInnerHTML e ele é literal**

`src/views/layout.tsx:59-65`

A varredura em src/views/ encontrou uma só ocorrência de dangerouslySetInnerHTML: o script inline de tema, uma string constante sem interpolação. Todos os dados de usuário passam pelo escape do preact-render-to-string.

OK **6. Realce de JSON por tokens, não por HTML montado à mão**

`src/shared/json-view.ts:1-8, 30-55`

A resposta do modelo (texto externo, não confiável) é exibida com destaque de sintaxe. O módulo devolve tokens tipados para o JSX escapar, em vez de string com — a decisão está escrita no cabeçalho do arquivo e evita exatamente a injeção que um dangerouslySetInnerHTML criaria.

OK **7. innerHTML no cliente só recebe HTML do próprio servidor; erro vai por textContent**

`src/client/transactions.ts:551-566, src/client/dividends.ts:41-47`

Os pontos que usam innerHTML (transactions.ts:552, dividends.ts:51) injetam fragmentos renderizados pelas views JSX, já escapados. As mensagens de erro — que podem vir do servidor em texto puro — são inseridas com textContent, com o motivo comentado em transactions.ts:560.

OK **8. Cor da classe validada por regex antes de virar atributo style**

`src/modules/allocation/allocation.schema.ts:6 e 15`

bucket.color é interpolado em style= em allocation.tsx:144 e :278. O schema exige /^#[0-9a-fA-F]{6}$/, então nada além de um hex chega lá — a injeção de CSS está barrada na fronteira, não só no escape.

OK **9. Download de nota sem path traversal nem injeção de cabeçalho**

`src/modules/broker-note/broker-note.service.ts:207-218, transaction.routes.ts:59-65`

O nome do arquivo é gerado pelo servidor a partir da data e do id (`${data}_${id}.${ext}`), nunca vindo da URL nem do nome que o usuário enviou; a extensão sai de uma tabela fechada (EXTENSIONS, linhas 38-44). O Content-Disposition recebe esse nome gerado, então não há aspas nem CRLF de usuário no cabeçalho.

OK **10. Ticker codificado antes de entrar em URL externa**

`src/integrations/yahoo/yahoo.client.ts:14 e 159`

O host é constante e o ticker informado pelo usuário passa por encodeURIComponent, então não há como escapar do caminho para outro destino. Os clientes BCB e Tesouro usam URLs fixas.

OK **11. Gate de interface com verificação equivalente no servidor**

`src/modules/transaction/transaction.routes.ts:107 ↔`
`src/integrations/anthropic/anthropic.client.ts:110`

A aba de importação por nota some da tela quando a chave da Anthropic não está configurada. Testado por requisição direta a POST /transactions/parse-note ignorando a interface: o servidor responde 400 "APP_ANTHROPIC_API_KEY não configurada." O gate não vive só no navegador.

OK **12. Nenhum segredo no código, nos configs, no CI ou no histórico**

`.gitignore:22, .env.example:24-29, .github/workflows/ci.yml, deploy/big-cat.service`

.env está no .gitignore e nunca foi versionado (git log --all sobre o caminho não retorna commit). O .env.example traz apenas campos vazios. O CI não define segredo algum e a unit systemd não embute credencial. Busca em todos os commits por sk-ant-, AKIA e blocos PRIVATE KEY: nada. As únicas strings parecidas com credencial são fixtures de teste ('sk-teste' em anthropic.client.test.ts:44, 'segredo' em auth.service.test.ts:14).

OK **13. Upload limitado na entrada**

`src/app.ts:46`

@fastify/multipart registrado com fileSize de 20 MB e files: 1 — o endpoint de nota não vira porta de entrada para arquivo grande nem para envio múltiplo.

OK **14. Cookie httpOnly e SameSite=Lax**

`src/plugins/auth.ts:41-47`

httpOnly bloqueia leitura por script (contendo o impacto de um XSS) e SameSite=Lax é o que hoje impede que os POSTs de escrita sejam disparados autenticados a partir de outro site — é o substituto de fato do token anti-CSRF nesta aplicação.

OK **15. Sandbox do systemd bem apertado**

`deploy/big-cat.service:31-66`

NoNewPrivileges, ProtectSystem=strict com dois ReadWritePaths, CapabilityBoundingSet vazio, ProtectHome=tmpfs, RestrictNamespaces, SystemCallFilter=@system-service e UMask=0077. Se a aplicação for comprometida, o raio de alcance no host é pequeno.

Pontos fracos: os riscos centrais

Um interruptor de segurança que se desliga sozinho

A autenticação não é uma camada que falha em partes: ela está ligada ou desligada, e o que decide é uma variável vazia. Um controle de tudo-ou-nada cujo estado "nada" é o default e não interrompe o boot é o risco estrutural deste projeto (F1).

O escape de HTML tem uma saída lateral

Todo o resto do projeto renderiza por JSX, que escapa. O handler de erro não — e é exatamente por onde a entrada do usuário volta ao navegador declarada como HTML (F2).

Destino de navegação tratado como dado inócuo

O campo returnTo atravessa sete handlers sem validação. É um dado que o usuário controla e que decide para onde o navegador vai depois de uma operação bem-sucedida — o momento de maior confiança para um phishing (F3).

Uma senha só, sem fricção nenhuma na tentativa

Com credencial única, ausência de limite de tentativas não é detalhe operacional: é a diferença entre um segredo e um segredo enumerável em rede local (F4).

Achados detalhados

Agrupados por categoria, em ordem de severidade. Cada achado traz arquivo e linha exatos, o trecho do código, por que é explorável, a condição de explorabilidade e a correção sugerida.

Severidade	Arquivo:linha	Descrição
CRÍTICA	`src/config/env.ts:18`	**F1** — Senha em branco desliga a autenticação inteira, e nada no boot rejeita isso 4. Chaves expostas / default inseguro
ALTA	`src/plugins/errors.ts:27`	**F2** — Handler de erro devolve a mensagem crua com Content-Type text/html, sem escape 5. Inputs sem tratamento (XSS)
ALTA	`src/modules/asset/asset.routes.ts:184`	**F3** — Open redirect: o campo returnTo dos formulários vai direto para o Location 5. Inputs sem tratamento (destino controlado pelo usuário)
ALTA	`src/modules/auth/auth.routes.ts:19`	**F4** — POST /login não tem limite de tentativas, atraso nem bloqueio Mecanismo de autenticação (transversal às categorias 2 e 4)
MÉDIA	`src/plugins/auth.ts:41`	**F5** — Cookie de sessão sem Secure e respostas sem cabeçalhos de segurança 4. Configuração de segurança
BAIXA	`src/plugins/errors.ts:11`	**F6** — Falha de validação vira 500 e devolve a estrutura interna do Zod 5. Tratamento de erro
INFORMATIVA	`src/modules/transaction/transaction.routes.ts:197`	**F7** — Rotas por id não têm verificação de posse — correto hoje, quebra no dia do multiusuário 3. IDOR (nota prospectiva)

F7 é informativa e prospectiva: não é falha na arquitetura atual. Está registrada porque é a superfície que vira IDOR imediatamente se o projeto ganhar uma segunda identidade.

CRÍTICA F1 — Senha em branco desliga a autenticação inteira, e nada no boot rejeita isso

Categoria: 4. Chaves expostas / default inseguro

Evidência

src/config/env.ts:18

```
APP_AUTH_PASSWORD: z.string().default(''),
```

src/modules/auth/auth.service.ts:46

```
this.authEnabled = options.password.length > 0
```

src/modules/auth/auth.service.ts:74

```
if (!this.authEnabled) return true   // isValid()
```

src/plugins/auth.ts:22

```
if (!auth.authEnabled) return   // hook onRequest
```

.env.example:24

```
APP_AUTH_PASSWORD=
```

src/main.ts:34

```
await app.listen({ port: env.PORT, host: '0.0.0.0' })
```

Por que é explorável

APP_AUTH_PASSWORD tem default de string vazia no schema Zod, e string vazia significa "autenticação desligada" em três camadas: o AuthService marca authEnabled como false (auth.service.ts:46), isValid() passa a devolver true para qualquer token — inclusive nenhum (auth.service.ts:74) — e o hook onRequest retorna antes de qualquer verificação (plugins/auth.ts:22). O loadEnv() valida formato mas não rejeita o valor vazio, nem mesmo com NODE_ENV=production, então o boot segue normalmente: o único sinal é um logger.warn (auth.service.ts:49) que, em produção, vai para o journal e ninguém lê. O .env.example distribui exatamente esse valor perigoso na linha 24 — copiar o exemplo e subir é o caminho documentado no README de deploy. E o listen é em 0.0.0.0 (main.ts:34), com o deploy/README.md:100 instruindo a acessar por http://<ip-da-maquina>:8001. O resultado é a carteira inteira — transações, proventos, patrimônio, PDFs de notas de corretagem com CPF e dados bancários — legível e gravável por qualquer host da rede, sem credencial nenhuma.

Verificação

O próprio teste do projeto documenta o comportamento: tests/integration/auth.test.ts:160-176, bloco "sem senha configurada", assevera que GET /assets/ responde 200 sem cookie. Confirmado nesta auditoria em app.inject: com APP_AUTH_PASSWORD='' todas as rotas abrem.

Condição de explorabilidade

Ativo sempre que APP_AUTH_PASSWORD estiver ausente ou vazia no ambiente. Não depende de flag nem de outra configuração.

Correção sugerida

Tornar a senha obrigatória quando NODE_ENV=production, via superRefine no EnvSchema, e exigir tamanho mínimo. Se o modo aberto tiver de continuar existindo para desenvolvimento, torná-lo explícito (APP_AUTH_DISABLED=true) em vez de implícito num campo vazio — assim ninguém desliga a autenticação por omissão.

ALTA F2 — Handler de erro devolve a mensagem crua com Content-Type text/html, sem escape

Categoria: 5. Inputs sem tratamento (XSS)

Evidência

src/plugins/errors.ts:27

```
return reply.code(status).type('text/html; charset=utf-8').send(error.message)
```

src/plugins/errors.ts:24

```
return reply.code(status).header('HX-Retarget', '#error-banner').send(error.message)
```

src/modules/allocation/asset-class.service.ts:27

```
throw HttpError.conflict(`Já existe uma classe chamada '${form.name}'.`)
```

src/modules/allocation/asset-class.service.ts:37

```
throw HttpError.conflict(`Já existe uma classe chamada '${form.name}'.`)
```

src/integrations/anthropic/anthropic.client.ts:115

```
throw HttpError.badRequest(`Tipo de arquivo não suportado: ${contentType}`)
```

Por que é explorável

O setErrorHandler é o ponto único de tradução de erro para resposta, e no ramo HTML ele envia error.message declarando text/html — o navegador então interpreta a string como marcação. Toda a defesa de XSS do projeto está nas views JSX, que escapam por padrão; este caminho não passa por view nenhuma. Basta que uma mensagem de erro carregue texto vindo do usuário, e três carregam: asset-class.service.ts:27 e :37 interpolam o nome da classe informado no formulário, e anthropic.client.ts:115 interpola o Content-Type do upload. O nome da classe é persistido, então a mensagem se repete a cada nova tentativa de cadastro com o mesmo nome.

Verificação

Reproduzido em app.inject nesta auditoria. POST /allocation/classes/new com name=<svg onload=alert(1)> (dentro do limite de 40 caracteres do schema) duas vezes:
1ª → 302 (classe criada)
2ª → 409, content-type: text/html; charset=utf-8
corpo: Já existe uma classe chamada '<svg onload=alert(1)>'.
O payload volta literal, sem <. Com a mesma sonda, um name de 41+ caracteres retorna "Too big: expected string to have <=40 characters" — o limite reduz o payload, não o impede.

Condição de explorabilidade

Com autenticação ligada, a exploração é limitada a self-XSS: o SameSite=Lax do cookie impede que um POST cross-site chegue autenticado à rota. Com a autenticação desligada (F1), um formulário em site de terceiro pode fazer POST para /allocation/classes/new e o navegador renderiza a resposta na origem da aplicação — XSS pleno, encadeado a partir de F1.

Correção sugerida

No ramo HTML e no ramo HTMX do setErrorHandler, escapar a mensagem (ou responder text/plain, que já resolve sem tocar em template). Em paralelo, parar de interpolar entrada do usuário em mensagem de erro: a mensagem diz o que houve, a interface é que mostra o valor — e a interface escapa.

ALTA

F3 — Open redirect: o campo returnTo dos formulários vai direto para o Location

Categoria: 5. Inputs sem tratamento (destino controlado pelo usuário)

Evidência

src/modules/asset/asset.routes.ts:27-30

```
function destinationOf(returnTo) { ... return trimmed === '' ? '/assets/' : trimmed }
```

src/modules/asset/asset.routes.ts:184

```
return reply.redirect(withPriceWarning(destinationOf(form.returnTo), warn), 302)
```

src/modules/transaction/transaction.routes.ts:134

```
return reply.redirect(form.returnTo ?? '/transactions/', 302)
```

src/modules/transaction/transaction.routes.ts:158

```
return reply.redirect(form.returnTo ?? '/transactions/', 302)
```

src/modules/transaction/transaction.routes.ts:165

```
return reply.redirect(returnTo ?? '/transactions/', 302)
```

src/modules/dividend/dividend.routes.ts:71

```
return reply.redirect(form.returnTo ?? '/dividends/', 302)
```

src/modules/dividend/dividend.routes.ts:85

```
return reply.redirect(form.returnTo ?? '/dividends/', 302)
```

src/modules/dividend/dividend.routes.ts:91

```
return reply.redirect(returnTo ?? '/dividends/', 302)
```

Por que é explorável

Sete handlers aceitam returnTo do corpo do formulário e o entregam ao reply.redirect() sem nenhuma verificação de que é um caminho interno. O schema declara o campo apenas como z.string().optional() (transaction.schema.ts, dividend.schema.ts, asset.routes.ts:70), então qualquer URL absoluta passa. O comentário em asset.routes.ts:19-26 mostra que o campo já deu problema antes — mas o conserto tratou a string vazia, não o destino externo. Um link para /transactions/123/delete com returnTo=https://... apontando para uma cópia da tela de login devolve o usuário, depois de uma ação legítima e bem-sucedida na aplicação, a um site controlado pelo atacante: o contexto de "acabei de operar no sistema" é exatamente o que torna o phishing de senha convincente.

Verificação

Reproduzido em app.inject nesta auditoria, com sessão válida:
POST /transactions/<id>/delete returnTo=https://evil.example.com/phish
→ 302 Location: https://evil.example.com/phish
POST /assets/PETR4/edit returnTo=//evil.example.com/
→ 302 Location: //evil.example.com/
POST /dividends/new returnTo=https://evil.example.com/
→ 302 Location: https://evil.example.com/
A forma //host é a mais traiçoeira: parece caminho relativo e o navegador a lê como URL protocol-relative.

Condição de explorabilidade

Exige que o usuário submeta o formulário — na prática, um link ou página preparada. Note que o redirect acontece DEPOIS da escrita no banco: mesmo com SameSite bloqueando o cookie num POST cross-site, a variante em que o próprio usuário é induzido a colar/seguir um link dentro da aplicação continua valendo.

Correção sugerida

Validar returnTo como caminho interno antes de redirecionar: aceitar apenas strings que comecem com uma única barra e não com '//' nem '/\'. Um helper em shared/ usado pelos sete pontos, com teste que fixe os casos https://, //host e /\host.

ALTA F4 — POST /login não tem limite de tentativas, atraso nem bloqueio

Categoria: Mecanismo de autenticação (transversal às categorias 2 e 4)

Evidência

`src/modules/auth/auth.routes.ts:19-31`

`app.post('/login', ...)  — sem contador, sem atraso, sem bloqueio`

`src/config/env.ts:19`

`APP_AUTH_SESSION_DAYS: z.coerce.number().int().positive().default(365)`

`package.json:—`

`sem @fastify/rate-limit nas dependências`

Por que é explorável

A aplicação tem um segredo só, e ele protege tudo. O handler de login compara a senha e responde 401, sem contabilizar tentativas por IP, sem atraso progressivo e sem bloqueio temporário; não há @fastify/rate-limit nem equivalente no projeto. Como o serviço escuta em 0.0.0.0, qualquer host da rede pode tentar senhas na velocidade que o servidor aguentar. A comparação em tempo constante (auth.service.ts:124-129) protege contra ataque de temporização, mas não contra volume — são problemas diferentes. E o sucesso é duradouro: a sessão padrão vale 365 dias (env.ts:19), então um acerto único dá um ano de acesso.

Verificação

Reproduzido em app.inject nesta auditoria: 30 POSTs consecutivos para /login com senhas erradas. Todos responderam 401 — nenhum 429, nenhum atraso perceptível, nenhum bloqueio. O contraste está em tests/integration/auth.test.ts:97-107, que cobre a senha errada uma vez, mas não a repetição.

Condição de explorabilidade

Sem pré-condição. Vale sempre que a autenticação estiver ligada — e é justamente o cenário 'protegido'.

Correção sugerida

Registrar @fastify/rate-limit só na rota de login (algo como 5 tentativas por minuto por IP, com bloqueio crescente), e reduzir APP_AUTH_SESSION_DAYS para uma janela defensável. Um contador de falhas em memória no AuthService, com pausa após N erros, já muda a ordem de grandeza do ataque.

MÉDIA F5 — Cookie de sessão sem Secure e respostas sem cabeçalhos de segurança

Categoria: 4. Configuração de segurança

Evidência

`src/plugins/auth.ts:41-47`

`return { httpOnly: true, path: '/', sameSite: 'lax', maxAge }  — sem `secure``

`src/app.ts:32-55`

`buildApp() não registra helmet nem define CSP/X-Frame-Options/X-Content-Type-Options`

`deploy/README.md:100`

```
Acesse em http://<ip-da-maquina>:8001 (bind 0.0.0.0)
```

src/views/layout.tsx:68-76

```
3 CDNs (jsdelivr, unpkg) carregadas sem atributo integrity
```

Por que é explorável

Três lacunas de configuração que se somam. (a) O cookie de sessão não tem o atributo Secure, e o deploy documentado serve HTTP puro na LAN: o token de 365 dias trafega em claro em toda requisição, visível para qualquer um que capture o tráfego do segmento — roubá-lo dá acesso completo sem precisar da senha. (b) Nenhuma resposta traz Content-Security-Policy, X-Content-Type-Options, X-Frame-Options ou Referrer-Policy (verificado nos cabeçalhos reais de GET /login: só content-type, content-length, date e connection). Uma CSP teria contido o impacto do F2. (c) As três tags de CDN em layout.tsx:68-76 não usam integrity/SRI, então a aplicação executa, com privilégio total de origem, o que jsdelivr e unpkg entregarem.

Verificação

Cabeçalhos de GET /login capturados nesta auditoria: {content-type, content-length, date, connection}. Nenhum cabeçalho de segurança. O Set-Cookie do login contém HttpOnly (asseverado em tests/integration/auth.test.ts:94) e não contém Secure.

Condição de explorabilidade

O item (a) exige um atacante no mesmo segmento de rede — realista num deploy de LAN doméstica com outros dispositivos. Os itens (b) e (c) são redução de defesa em profundidade, não falha explorável sozinha.

Correção sugerida

Ligar `secure: true` quando NODE_ENV=production (e pôr a aplicação atrás de TLS, ainda que com certificado próprio); registrar @fastify/helmet com CSP restritiva; e fixar SRI nas três tags de CDN — ou passar a servir os assets do próprio public/.

BAIXA F6 — Falha de validação vira 500 e devolve a estrutura interna do Zod

Categoria: 5. Tratamento de erro

Evidência

src/plugins/errors.ts:11

```
const status = isHttpError(error) ? error.statusCode : (error.statusCode ?? 500)
```

src/modules/dividend/dividend.routes.ts:61

```
const form = DividendForm.parse(req.body)   // ZodError sem statusCode → 500
```

src/modules/transaction/transaction.routes.ts:113

```
const form = TransactionForm.parse(req.body)
```

src/modules/allocation/allocation.routes.ts:71-77

```
parseForm() — o jeito certo, usado só na allocation
```

Por que é explorável

As rotas de allocation passam por um helper parseForm() que converte erro de Zod em 400 com mensagem curta (allocation.routes.ts:71-77). As demais chamam .parse() direto: o ZodError sobe sem statusCode, o handler classifica como 500, registra como "erro não tratado" (errors.ts:13) e devolve ao cliente o JSON completo das issues — caminho do campo, código interno e valor recebido. É ruído de diagnóstico exposto e log de erro poluído com o que na verdade é pedido malfeito; o próprio comentário em allocation.routes.ts:65-70 já diagnosticou o problema, mas a correção não foi propagada.

Verificação

Reproduzido nesta auditoria: POST /dividends/new com total_amount não numérico → 500 e corpo com o array de issues do Zod, incluindo "received": "NaN" e o path do campo.

Condição de explorabilidade

Requer sessão válida. Sem impacto direto de confidencialidade neste projeto — é higiene de erro e ruído operacional.

Correção sugerida

Promover o parseForm() de allocation.routes.ts para shared/ e usá-lo em todas as rotas, ou tratar ZodError no setErrorHandler convertendo para 400 com a primeira mensagem.

INFORMATIVA F7 — Rotas por id não têm verificação de posse — correto hoje, quebra no dia do multiusuário

Categoria: 3. IDOR (nota prospectiva)

Evidência

`src/modules/transaction/transaction.routes.ts:197-203`

`GET /transactions/notes/:id — serve o PDF de qualquer nota pelo id`

`src/modules/broker-note/broker-note.service.ts:167-171`

`findNote(id) — busca por id, sem cláusula de dono`

`src/modules/dividend/dividend.routes.ts:74/88`

`updateDividend(Number(id)) / deleteDividend(Number(id))`

Por que é explorável

Não é falha: o sistema tem um dono só, e o portão de sessão já decide tudo. Fica registrado porque /transactions/notes/:id serve PDFs de notas de corretagem — o dado mais sensível do acervo — por id sequencial e sem nenhuma condição além de estar autenticado. No dia em que existir uma segunda identidade, estes são os handlers que viram IDOR imediatamente, e nenhum teste hoje falharia para avisar.

Verificação

Leitura dos 58 handlers. Nenhuma cláusula de posse existe porque nenhuma coluna de dono existe (prisma/schema.prisma, 11 modelos).

Condição de explorabilidade

Não explorável na arquitetura atual.

Correção sugerida

Se multiusuário entrar no roadmap, introduzir a coluna de dono junto com um helper de busca que exija o id do dono — não depois.

Recomendações priorizadas

A ordem reflete impacto sobre esforço. As duas de P1 fecham a exposição total da base e o único caminho de XSS confirmado, e nenhuma passa de algumas dezenas de linhas.

Prio	Ação	Achado	Por quê
P1	**Exigir senha em produção e falhar o boot sem ela**	F1	É a única falha que expõe a base inteira sem nenhuma condição prévia, e é também o multiplicador do F2. Um superRefine no EnvSchema resolve em poucas linhas.
P1	**Escapar (ou mandar como text/plain) a mensagem no handler de erro**	F2	Corrige o único caminho de XSS confirmado, num arquivo só, sem tocar em nenhuma view.
P2	**Validar returnTo como caminho interno nos sete redirects**	F3	Um helper em shared/ e um teste que fixe https://, //host e /\host.
P2	**Limitar tentativas de login e encurtar a sessão**	F4	@fastify/rate-limit só na rota de login, mais um valor de APP_AUTH_SESSION_DAYS menor que 365.
P3	**Secure no cookie, helmet com CSP e SRI nas CDNs**	F5	Defesa em profundidade: reduz o impacto de qualquer XSS futuro e tira o token de sessão do tráfego em claro.
P3	**Propagar o parseForm() para todas as rotas**	F6	Higiene: erro de validação vira 400 com mensagem curta, e o log de erro volta a significar erro de verdade.

Uma observação de sequência: corrigir F1 sem corrigir F2 deixa o XSS refletido como self-XSS — aceitável. Corrigir F2 sem corrigir F1 não adianta quase nada, porque a base continua aberta a quem estiver na rede. Se só houver fôlego para uma, comece por F1.

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar. Achados triviais do mesmo tema foram agrupados numa issue só — a issue 5 reúne cookie, cabeçalhos e SRI; a issue 3 reúne os sete redirects — para não gerar spam. F7 não vira issue por ser informativa e não acionável hoje.

CRÍTICA **ISSUE 1** — labels: `security`, `severity:crítica`, `auth`

```
--- ISSUE 1 ---

# [Segurança] Senha em branco desliga a autenticação inteira sem falhar o boot

Labels: `security`, `severity:crítica`, `auth`

## Problema

`APP_AUTH_PASSWORD` tem default de string vazia no schema de ambiente, e string vazia
significa **autenticação desligada** em três camadas encadeadas:

1. `AuthService` marca `authEnabled = false` (`auth.service.ts:46`);
2. `isValid()` passa a devolver `true` para qualquer token, inclusive nenhum
  (`auth.service.ts:74`);
3. o hook `onRequest` retorna antes de qualquer verificação (`plugins/auth.ts:22`).

`loadEnv()` valida o formato mas **não rejeita o valor vazio**, nem com `NODE_ENV=production`.
O boot segue normal; o único sinal é um `logger.warn` que, em produção, vira uma linha no
journal que ninguém lê.

### Por que é explorável

O `.env.example` distribui exatamente esse valor perigoso (linha 24) e copiar o exemplo é o
caminho documentado em `deploy/README.md`. O `listen` é em `0.0.0.0` (`main.ts:34`) e o próprio
README manda acessar por `http://<ip-da-maquina>:8001`. Resultado: transações, proventos,
patrimônio e os PDFs de notas de corretagem — com CPF e dados bancários — ficam legíveis e
graváveis por qualquer host da rede, sem credencial nenhuma. Não há pré-condição: basta a
variável estar ausente.

## Evidência

`src/config/env.ts:18`
```ts
APP_AUTH_PASSWORD: z.string().default(''),
```

`src/modules/auth/auth.service.ts:46,74`
```ts
this.authEnabled = options.password.length > 0
// ...
isValid(token: string | null | undefined): boolean {
  if (!this.authEnabled) return true
```

`src/plugins/auth.ts:22`
```ts
app.addHook('onRequest', async (request, reply) => {
  if (!auth.authEnabled) return
```

O comportamento já está fixado por teste em `tests/integration/auth.test.ts:160-176`
("sem senha configurada" → `GET /assets/` responde 200 sem cookie).

Impacto

Exposição total da base de dados financeira e dos arquivos de nota a qualquer um com acesso
de rede à porta. É também o multiplicador da issue de XSS refletido: com autenticação
desligada, aquele self-XSS vira XSS pleno explorável cross-site.

Correção sugerida

Em `src/config/env.ts`, um `superRefine` que rejeite senha vazia quando
`NODE_ENV === 'production'` e exija tamanho mínimo. Se o modo aberto precisa continuar
existindo para desenvolvimento, torná-lo **explícito** (`APP_AUTH_DISABLED=true`) em vez de
implícito num campo vazio — ninguém deve desligar autenticação por omissão.

Critérios de aceite
```

```
- [ ] `loadEnv()` lança erro quando `NODE_ENV=production` e `APP_AUTH_PASSWORD` está vazia
- [ ] Senha aceita apenas com tamanho mínimo definido (sugestão: 12 caracteres)
- [ ] Desligar a autenticação exige uma variável explícita, nunca um campo em branco
- [ ] `.env.example` documenta que a senha é obrigatória em produção
- [ ] Teste cobrindo: produção + senha vazia derruba o boot; produção + senha válida sobe
- [ ] Teste existente de "sem senha configurada" atualizado para o novo contrato explícito

--- FIM ISSUE 1 ---
```

ALTA **ISSUE 2** — labels: `security`, `severity:alta`, `xss`

```
--- ISSUE 2 ---

# [Segurança] XSS refletido: handler de erro devolve mensagem crua como text/html

Labels: `security`, `severity:alta`, `xss`

## Problema

O `setErrorHandler` é o ponto único de tradução de erro para resposta, e no ramo HTML ele
envia `error.message` declarando `text/html` — o navegador interpreta a string como marcação.
Toda a defesa de XSS do projeto está nas views JSX, que escapam por padrão; **este caminho não
passa por view nenhuma**.

### Por que é explorável

Basta que uma mensagem de erro carregue texto vindo do usuário, e três carregam:
`asset-class.service.ts:27` e `:37` interpolam o nome da classe informado no formulário, e
`anthropic.client.ts:115` interpola o `Content-Type` do upload. O nome da classe é persistido,
então a mensagem se repete a cada nova tentativa de cadastro com o mesmo nome.

Com autenticação ligada, o `SameSite=Lax` do cookie limita o alcance a self-XSS. **Com a
autenticação desligada** (ver a issue de senha em branco), um formulário em site de terceiro
faz `POST` para `/allocation/classes/new` e o navegador renderiza a resposta **na origem da
aplicação** — XSS pleno.

## Evidência

`src/plugins/errors.ts:24,27`
```ts
if (request.headers['hx-request']) {
  return reply.code(status).header('HX-Retarget', '#error-banner').send(error.message)
}
return reply.code(status).type('text/html; charset=utf-8').send(error.message)
```

`src/modules/allocation/asset-class.service.ts:27` (e idêntico em `:37`)
```ts
throw HttpError.conflict(`Já existe uma classe chamada '${form.name}'.`)
```

Reproduzido com `app.inject` — `POST /allocation/classes/new` com
`name=<svg onload=alert(1)>`, duas vezes:

```
1ª → 302 (classe criada)
2ª → 409  content-type: text/html; charset=utf-8
     corpo: Já existe uma classe chamada '<svg onload=alert(1)>'.
```

O payload volta literal, sem `<`. O limite de 40 caracteres do schema encurta o payload,
não o impede.

Impacto

Execução de script na origem da aplicação. Como o cookie é `httpOnly`, o roubo direto de
sessão está contido, mas o script age como o usuário: criar/apagar transações e proventos,
ler o histórico inteiro e baixar os PDFs de notas de corretagem.

Correção sugerida

Escapar a mensagem nos ramos HTML e HTMX do `setErrorHandler` — ou, mais simples, responder
`text/plain`, que resolve sem tocar em template algum. Em paralelo, parar de interpolar
entrada do usuário em mensagem de erro: a mensagem diz **o que** houve, e a interface, que
escapa, é quem mostra o valor.

Critérios de aceite
- [ ] Resposta de erro não interpreta HTML vindo da mensagem (escape ou `text/plain`)
```

````
- [ ] O mesmo vale para o ramo HTMX (`HX-Retarget`)
- [ ] `asset-class.service.ts:27` e `:37` deixam de interpolar `form.name` na mensagem
- [ ] Teste: conflito de nome com `<svg onload=alert(1)>` não devolve a tag literal em HTML
- [ ] Teste cobrindo o ramo HTMX com o mesmo payload

--- FIM ISSUE 2 ---
````

ALTA **ISSUE 3** — labels: `security`, `severity:alta`, `open-redirect`

````
--- ISSUE 3 ---

# [Segurança] Open redirect: returnTo vai direto para o Location em 7 endpoints

Labels: `security`, `severity:alta`, `open-redirect`

## Problema

Sete handlers aceitam `returnTo` do corpo do formulário e o entregam ao `reply.redirect()`
sem verificar que é um caminho interno. O schema declara o campo apenas como
`z.string().optional()`, então qualquer URL absoluta passa.

Os sete pontos:

| Arquivo | Linha | Rota |
|---|---|---|
| `src/modules/asset/asset.routes.ts` | 184 (via `destinationOf`, 27-30) | `POST
  /assets/:ticker/edit` |
| `src/modules/transaction/transaction.routes.ts` | 134 | `POST /transactions/new` |
| `src/modules/transaction/transaction.routes.ts` | 158 | `POST /transactions/:id/edit` |
| `src/modules/transaction/transaction.routes.ts` | 165 | `POST /transactions/:id/delete` |
| `src/modules/dividend/dividend.routes.ts` | 71 | `POST /dividends/new` |
| `src/modules/dividend/dividend.routes.ts` | 85 | `POST /dividends/:id/edit` |
| `src/modules/dividend/dividend.routes.ts` | 91 | `POST /dividends/:id/delete` |

### Por que é explorável

O redirect acontece **depois** de uma ação legítima e bem-sucedida. O usuário apaga uma
transação, a operação funciona, e ele é despejado numa cópia da tela de login hospedada pelo
atacante — o contexto de "acabei de operar no sistema" é justamente o que torna o phishing de
senha convincente. A forma `//host` é a mais traiçoeira: parece caminho relativo e o navegador
a lê como URL protocol-relative.

O comentário em `asset.routes.ts:19-26` mostra que o campo já deu problema antes, mas o
conserto tratou a string vazia, não o destino externo.

## Evidência

`src/modules/asset/asset.routes.ts:27-30,184`
```ts
function destinationOf(returnTo: string | undefined): string {
  const trimmed = returnTo?.trim() ?? ''
  return trimmed === '' ? '/assets/' : trimmed
}
// ...
return reply.redirect(withPriceWarning(destinationOf(form.returnTo), warn), 302)
```

`src/modules/transaction/transaction.routes.ts:165`
```ts
return reply.redirect(returnTo ?? '/transactions/', 302)
```

Reproduzido com `app.inject`, com sessão válida:

```
POST /transactions/<id>/delete  returnTo=https://evil.example.com/phish
  → 302  Location: https://evil.example.com/phish
POST /assets/PETR4/edit         returnTo=//evil.example.com/
  → 302  Location: //evil.example.com/
POST /dividends/new             returnTo=https://evil.example.com/
  → 302  Location: https://evil.example.com/
```

## Impacto

Phishing de credencial com a autoridade do domínio da aplicação, no momento de maior
confiança do usuário. Também serve de trampolim para levar a vítima a qualquer destino
externo a partir de um link que aparenta ser interno.
````

```
## Correção sugerida

Um helper em `src/shared/` que aceite apenas caminho interno — começa com uma barra e **não**
com `//` nem `/\` — e devolva o default quando não passar. Usá-lo nos sete pontos.

## Critérios de aceite

- [ ] Helper de validação de destino interno criado em `src/shared/`
- [ ] Os sete redirects listados passam por ele
- [ ] Teste fixando os casos rejeitados: `https://host`, `//host`, `/\host`, `http:/host`
- [ ] Teste fixando os casos aceitos: `/assets/`, `/transactions/?ticker=PETR4`, vazio → default
- [ ] Destino inválido cai no default, sem erro visível ao usuário

--- FIM ISSUE 3 ---
```

ALTA **ISSUE 4** — labels: `security`, `severity:alta`, `auth`

```
--- ISSUE 4 ---

# [Segurança] Login sem limite de tentativas e sessão de 365 dias

Labels: `security`, `severity:alta`, `auth`

## Problema

A aplicação tem **um segredo só**, e ele protege tudo. O handler de login compara a senha e
responde 401 — sem contabilizar tentativas por IP, sem atraso progressivo, sem bloqueio
temporário. Não há `@fastify/rate-limit` nem equivalente nas dependências.

### Por que é explorável

O serviço escuta em `0.0.0.0` (`main.ts:34`), então qualquer host da rede pode tentar senhas na
velocidade que o servidor aguentar. A comparação em tempo constante
(`auth.service.ts:124-129`) protege contra ataque de temporização, mas não contra volume — são
problemas diferentes. E o sucesso é duradouro: a sessão padrão vale **365 dias**
(`env.ts:19`), então um acerto único dá um ano de acesso.

## Evidência

`src/modules/auth/auth.routes.ts:19-31`
```ts
app.post('/login', async (req, reply) => {
  const { password } = LoginForm.parse(req.body)
  const token = c.auth.login(password)

  if (token === null) {
    return reply.code(401).html(LoginPage({ error: true }))
  }
  // ... sem contador, sem atraso, sem bloqueio
})
```

`src/config/env.ts:19`
```ts
APP_AUTH_SESSION_DAYS: z.coerce.number().int().positive().default(365),
```

Reproduzido com `app.inject`: 30 POSTs consecutivos para `/login` com senhas erradas. Todos
responderam 401 — nenhum 429, nenhum atraso, nenhum bloqueio.

Impacto

Ataque de força bruta contra a única credencial do sistema, na velocidade da rede local.
Acertar uma vez concede um ano de acesso a toda a carteira.

Correção sugerida

Registrar `@fastify/rate-limit` **apenas** na rota de login (sugestão: 5 tentativas por minuto
por IP, com bloqueio crescente) e reduzir `APP_AUTH_SESSION_DAYS` para uma janela defensável.
Um contador de falhas em memória no `AuthService`, com pausa após N erros, já muda a ordem de
grandeza do ataque.

Critérios de aceite

- [ ] `POST /login` limitado por IP, devolvendo 429 ao estourar
- [ ] Tentativas malsucedidas registradas em log (sem a senha tentada)
- [ ] `APP_AUTH_SESSION_DAYS` com default reduzido e documentado no `.env.example`
- [ ] Teste: N+1 tentativas erradas seguidas devolvem 429
```

```
- [ ] Teste: o limite não afeta um login legítimo após o intervalo

--- FIM ISSUE 4 ---
```

MÉDIA **ISSUE 5** — labels: `security`, `severity:média`, `hardening`

```
--- ISSUE 5 ---

# [Segurança] Cookie sem Secure, respostas sem cabeçalhos de segurança e CDNs sem SRI

Labels: `security`, `severity:média`, `hardening`

## Problema

Três lacunas de configuração agrupadas nesta issue por serem do mesmo tema — endurecimento da
camada HTTP.

**(a) Cookie de sessão sem `Secure`.** `sessionCookieOptions` define `httpOnly`, `path` e
`sameSite`, mas não `secure`. O deploy documentado serve HTTP puro na LAN
(`deploy/README.md:100`), então o token — válido por 365 dias — trafega em claro em toda
requisição. Quem capturar o tráfego do segmento e roubar o cookie entra sem precisar da senha.

**(b) Nenhum cabeçalho de segurança.** `buildApp()` não registra helmet nem define
`Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options` ou `Referrer-Policy`.
Uma CSP teria contido o impacto do XSS refletido reportado em issue separada.

**(c) CDNs sem SRI.** As três tags externas de `layout.tsx:68-76` (jsdelivr e unpkg) não têm
atributo `integrity`, então a aplicação executa com privilégio total de origem o que esses
hosts entregarem.

## Evidência

`src/plugins/auth.ts:41-47`
```ts
export function sessionCookieOptions(maxAgeSeconds: number) {
  return {
    httpOnly: true,
    path: '/',
    sameSite: 'lax' as const,
    maxAge: maxAgeSeconds,
  }   // sem `secure`
}
```

Cabeçalhos reais de `GET /login`, capturados na auditoria:
```
content-type, content-length, date, connection
```
Nenhum cabeçalho de segurança.

`src/views/layout.tsx:68-76` — três `<link>`/`<script>` para CDN, nenhum com `integrity`.

Impacto

(a) Roubo de sessão por quem estiver no mesmo segmento de rede — realista numa LAN doméstica
com outros dispositivos. (b) e (c) são redução de defesa em profundidade: nenhum é explorável
sozinho, mas ambos ampliam o estrago de qualquer XSS ou comprometimento de CDN.

Correção sugerida

- `secure: true` quando `NODE_ENV=production`, com a aplicação atrás de TLS (certificado
  próprio serve num deploy de LAN);
- registrar `@fastify/helmet` com CSP restritiva;
- fixar `integrity` nas três tags de CDN, ou passar a servir esses assets do próprio `public/`.

Critérios de aceite

- [ ] Cookie de sessão sai com `Secure` em produção
- [ ] Respostas trazem CSP, `X-Content-Type-Options`, `X-Frame-Options` e `Referrer-Policy`
- [ ] A CSP não quebra o HTMX nem o Chart.js (verificado com a aplicação rodando)
- [ ] As três tags de CDN têm `integrity` + `crossorigin`, ou os assets são locais
- [ ] Teste asseverando `Secure` no `Set-Cookie` com `NODE_ENV=production`

--- FIM ISSUE 5 ---
```

BAIXA **ISSUE 6** — labels: `security`, `severity:baixa`, `dx`

````
--- ISSUE 6 ---

# [Segurança] Erro de validação vira 500 e devolve a estrutura interna do Zod

Labels: `security`, `severity:baixa`, `dx`

## Problema

As rotas de allocation passam por um helper `parseForm()` que converte erro de Zod em 400 com
mensagem curta (`allocation.routes.ts:71-77`). **As demais chamam `.parse()` direto**: o
`ZodError` sobe sem `statusCode`, o handler classifica como 500, registra como "erro não
tratado" (`errors.ts:13`) e devolve ao cliente o JSON completo das issues — caminho do campo,
código interno e valor recebido.

O comentário em `allocation.routes.ts:65-70` já diagnosticou o problema; a correção só não foi
propagada para os outros módulos.

## Evidência

`src/plugins/errors.ts:11`
```ts
const status = isHttpError(error) ? error.statusCode : (error.statusCode ?? 500)
```

`src/modules/dividend/dividend.routes.ts:61` (mesmo padrão em `transaction.routes.ts:113`,
`:138`, e nas rotas `/api`)
```ts
const form = DividendForm.parse(req.body)
```

Reproduzido: `POST /dividends/new` com `total_amount` não numérico →
```
500
[ { "expected": "number", "code": "invalid_type", "received": "NaN",
    "path": ["total_amount"], "message": "Invalid input: expected number, received NaN" } ]
```

## Impacto

Sem impacto direto de confidencialidade neste projeto — é higiene de erro. O custo real é
operacional: o log de erro enche de "erro não tratado" para o que na verdade é pedido
malfeito, e um 500 esconde que o problema é do cliente, atrapalhando o diagnóstico do
problema seguinte.

## Correção sugerida

Promover o `parseForm()` de `allocation.routes.ts` para `src/shared/` e usá-lo em todas as
rotas; ou tratar `ZodError` no próprio `setErrorHandler`, convertendo para 400 com a primeira
mensagem.

## Critérios de aceite

- [ ] Falha de validação responde 400, não 500, em todas as rotas
- [ ] A resposta traz mensagem curta, não o array de issues do Zod
- [ ] Requisição malfeita registra `warn`, não `error`, no log
- [ ] Teste cobrindo pelo menos uma rota de formulário e uma rota `/api`

--- FIM ISSUE 6 ---
````